

Proyecto Final MITEX: Sistema de Soporte Inteligente y Recomendación Conversacional de Recetas CulinaryRAG

Curso 2025/2026

Índice

1. Introducción y Objetivos	3
2. Diseño Arquitectónico Global	3
2.1. Pipeline Offline (Pre-procesamiento)	3
2.2. Pipeline Online (Bucle en Tiempo Real)	5
3. Detalles de Implementación	5
3.1. Procesamiento e Ingestión Eficiente	5
3.2. Búsqueda Híbrida RRF y Re-ranking Topic-Aware	6
3.3. Agente Router y Decodificación Restringida	6
3.4. Resumen Extractivo con TextRank	7
4. Discusión, Validación y Resultados	7
4.1. Validación de Modelado de Tópicos (Experimento de Semillas)	7
5. Conclusiones y Trabajo Futuro	8
5.1. Principales Logros del Sistema	8
5.2. Líneas de Trabajo Futuro	8

1. Introducción y Objetivos

El presente proyecto documenta el diseño, desarrollo y validación de **CulinaryRAG**, un asistente conversacional inteligente y recomendador especializado en el dominio culinario. El sistema está construido a partir del dataset público de Kaggle *Food.com Recsys Dataset*, el cual contiene más de 180,000 recetas e interacciones asociadas.

El principal desafío abordado es la superación de las limitaciones típicas de los recomendadores tradicionales (basados únicamente en filtrado colaborativo o coincidencia exacta de palabras clave), proporcionando una interfaz conversacional fluida que comprende las intenciones complejas del usuario (por ejemplo, preferencias dietéticas, restricciones de tiempo o negaciones del tipo "sin horno." "sin gluten").

Para lograr esto, CulinaryRAG integra de manera cohesiva diversas técnicas avanzadas de minería de textos:

- **Procesamiento de Lenguaje Natural (NLP)** para la limpieza y preparación masiva de datos.
- **Modelado de Tópicos (Topic Modeling)** mediante BERTopic en fase offline para categorizar y etiquetar semánticamente las recetas.
- **Búsqueda Híbrida Vectorial y Léxica** en tiempo real basada en ChromaDB y BM25, unificada mediante Reciprocal Rank Fusion (RRF).
- **Re-ranking Semántico** a través de un modelo Cross-Encoder para asegurar la precisión y manejar negaciones lógicas.
- **Resumen Extractivo** basado en el algoritmo de grafos TextRank para condensar reseñas históricas y alimentar la ventana de contexto de forma eficiente.
- **Orquestación Agéntica** mediante un Grafo Acíclico Dirigido (DAG) en tiempo real con decodificación restringida sobre un LLM local (*Gemma-3-1b-it*), incorporando un fallback dinámico a búsquedas web.

El sistema evita explícitamente el uso de frameworks de alto nivel opacos como LangChain o LlamaIndex, garantizando el control absoluto de cada etapa del pipeline y demostrando el dominio técnico de las herramientas fundamentales de minería de textos.

2. Diseño Arquitectónico Global

El sistema CulinaryRAG se divide lógicamente en dos grandes bloques operativos para garantizar la escalabilidad técnica y un tiempo de respuesta óptimo en producción: el **Pipeline Offline** (procesamiento y preparación) y el **Pipeline Online** (bucle de interacción en tiempo real).

2.1. Pipeline Offline (Pre-procesamiento)

Este bloque se ejecuta de forma asíncrona antes de la interacción con el usuario y realiza las tareas más pesadas computacionalmente:

1. **Ingestión y Filtro de Calidad** (`ingestion.py` / `preprocesado.py`): Lee los datasets originales, limpia caracteres especiales y etiquetas HTML, elimina registros nulos y aplica filtros estadísticos para conservar únicamente interacciones de alta calidad (usuarios activos con al menos 8 reseñas).

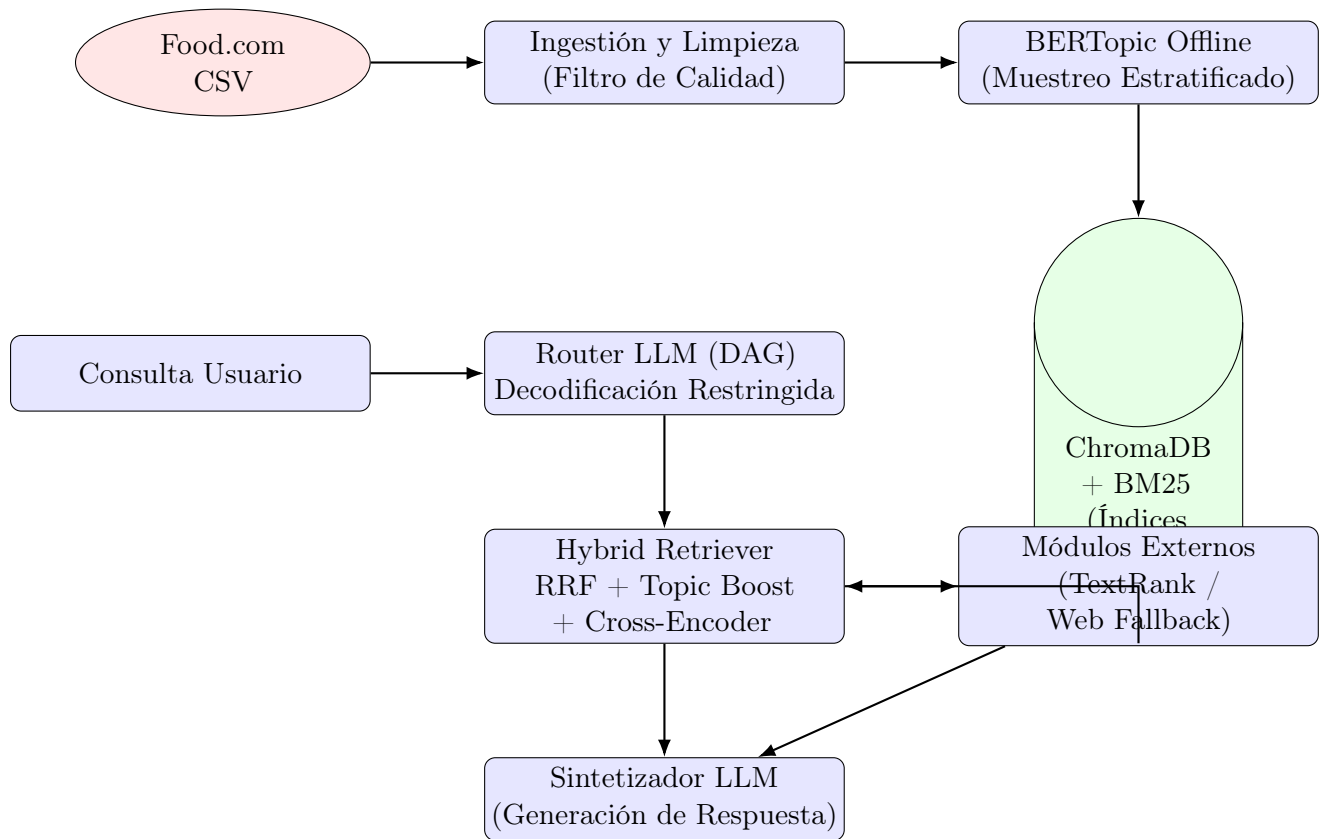


Figura 1: Diagrama de la Arquitectura de Bloques Global de CulinaryRAG.

2. **Modelado de Tópicos Estratificado (topics.py):** Extrae una muestra representativa de 10,000 recetas mediante un muestreo estratificado por tiempos de cocción. Entrena un modelo BERTopic con y sin semillas directivas para agrupar las recetas en estilos culinarios (vegana, repostería, desayunos, etc.).
3. **Indexación Semántica y Léxica (retriever.py):** Genera embeddings vectoriales ricos combinando el nombre, la descripción y los ingredientes de las recetas mediante el modelo transformer *paraphrase-multilingual-MiniLM-L12-v2* y los almacena de forma persistente en ChromaDB. En paralelo, construye un índice léxico basado en BM25 (`bm25_model.pkl`) a partir de la tokenización normalizada de los mismos textos.

2.2. Pipeline Online (Bucle en Tiempo Real)

Cuando el usuario introduce una consulta, el sistema la procesa instantáneamente a través de un flujo estructurado:

1. **Agente Orquestador DAG (agent.py):** El LLM actúa como router inteligente. Analiza la consulta y decide cuál es la mejor ruta mediante **decodificación restringida**. Esto obliga al LLM a emitir exactamente una de las acciones válidas: **recomendador** (búsqueda de recetas local), **resumidor** (síntesis de reseñas) o **web** (búsqueda externa).
2. **Buscador Híbrido Avanzado (retriever.py):** Si se activa el recomendador, se realiza una búsqueda paralela semántica (en ChromaDB) y léxica (con BM25). Las puntuaciones se unifican mediante la fórmula RRF.
3. **Re-ranking y Topic Boost:** Se clasifica el tópico de la consulta del usuario mediante BERTopic. Si una receta del ranking coincide con el tópico de la pregunta, recibe un **Topic Boost del 50 %**. Finalmente, los mejores candidatos se evalúan mediante un modelo **Cross-Encoder (ms-marco)** para calcular la relevancia contextual exacta.
4. **Generación y TextRank (summarizer.py / llm.py):** Si las recetas elegidas tienen exceso de reseñas, se aplica el algoritmo extractivo **TextRank** para inyectar solo las 3 frases más relevantes en el contexto del LLM. El modelo de lenguaje local sintetiza la respuesta y la presenta al usuario de forma natural.

3. Detalles de Implementación

Esta sección analiza las decisiones técnicas críticas tomadas durante el desarrollo de los módulos de CulinaryRAG, justificando su diseño en función de la robustez, el rendimiento y la flexibilidad.

3.1. Procesamiento e Ingestión Eficiente

La ingesta masiva se implementa en `preprocesado.py`. Para evitar la inclusión de datos irrelevantes o ruidosos que degraden los embeddings y sobrecarguen la base vectorial, se aplican filtros estrictos:

```
1 # Limpieza: Eliminar interacciones sin resena escrita
2 df_interactions = df_interactions.dropna(subset=['review'])
3
4 # Filtro de calidad: Usuarios con al menos 8 reseñas
5 reviews_por_user = df_interactions['user_id'].value_counts()
6 usuarios_def = reviews_por_user[reviews_por_user >= 8].index
```

```

7 df_interactions_filtrado = df_interactions[df_interactions['user_id'].isin(
    usuarios_def)].copy()

```

Listing 1: Filtro de calidad de interacciones y usuarios activos en preprocesado.py.

Este preprocesado asegura que la base de datos local contenga datos de alta fidelidad, vinculando recetas válidas con reseñas sustanciales y reduciendo la dimensionalidad del corpus antes del modelado de tópicos.

3.2. Búsqueda Híbrida RRF y Re-ranking Topic-Aware

El corazón de la recuperación se encuentra en `retriever.py`. Combina la recuperación semántica (que comprende la sinonimia y conceptos generales) con la recuperación léxica (crucial para marcas específicas de ingredientes o términos técnicos). La unificación se realiza mediante una implementación propia de **Reciprocal Rank Fusion (RRF)** con $k = 60$, mejorada dinámicamente con un multiplicador de tópico:

```

1 # Fusion RRF con Bonus por Topico
2 all_ids = set(rank_sem.keys()) | set(rank_lex.keys())
3 rrf_scores = []
4 for doc_id in all_ids:
5     score = 0.0
6     if doc_id in rank_sem:
7         score += 1.0 / (RRF_K + rank_sem[doc_id])
8     if doc_id in rank_lex:
9         score += 1.0 / (RRF_K + rank_lex[doc_id])
10
11 # BONUS DE TOPICO: Empuje del 50% si el topico de la receta coincide con el
12 # de la consulta
13 if query_topic != -1 and id_to_topic.get(doc_id) == query_topic:
14     score *= 1.5
15
16 rrf_scores.append((doc_id, score))

```

Listing 2: Implementación manual de RRF con Topic-Aware Boosting en retriever.py.

Esta fusión de rankings se complementa con un reordenamiento final mediante el Cross-Encoder `cross-encoder/ms-marco-MiniLM-L-6-v2`. A diferencia de los Bi-Encoders que comparan vectores precalculados de forma aislada, el Cross-Encoder realiza un procesamiento conjunto de la consulta y el documento a través de mecanismos de atención completa, capturando relaciones de negación lógica y cuantificadores esenciales (por ejemplo, distinguiendo perfectamente entre `chocolate con leche` y `chocolate sin leche`).

3.3. Agente Router y Decodificación Restringida

Para la orquestación, implementamos un router DAG basado en el LLM. Los frameworks tradicionales usan prompts de sistema con la esperanza de que el LLM devuelva un JSON o un comando estructurado, lo cual es propenso a fallos en modelos locales pequeños (1B a 3B). En CulinaryRAG, se utiliza **decodificación restringida** a nivel de token. Durante el paso del bucle donde el modelo debe emitir el nombre de la acción, restringimos los logits del vocabulario del tokenizador para permitir única y exclusivamente los tokens que conforman las herramientas válidas (`calculadora`, `recomendador`, `resumidor`, `web`), garantizando un comportamiento 100% determinista en la toma de decisiones agénticas.

3.4. Resumen Extractivo con TextRank

Cuando el recomendador recupera recetas populares con decenas de reseñas extensas, inyectar el texto completo en el contexto del LLM saturaría los tokens de entrada y provocaría el fenómeno del olvido o alucinaciones. El módulo `summarizer.py` implementa el algoritmo **TextRank**. Este enfoque modela las oraciones como nodos en un grafo, donde los arcos representan la similitud léxica o de solapamiento de palabras (normalizada por longitud). Tras aplicar el algoritmo de centralidad (PageRank), se extraen únicamente las N oraciones más centrales, logrando un resumen denso que conserva el consenso general de la comunidad sobre la receta.

4. Discusión, Validación y Resultados

Esta sección describe las pruebas analíticas e investigaciones empíricas diseñadas para validar las decisiones de arquitectura de CulinaryRAG frente a aproximaciones baseline.

4.1. Validación de Modelado de Tópicos (Experimento de Semillas)

Para categorizar recetas complejas en base a ingredientes y etiquetas, implementamos un experimento comparativo (`experimento.py`) para evaluar la madurez de **BERTopic** bajo dos configuraciones:

1. **Baseline No Supervisado (Sin Semillas):** El modelo agrupa libremente los documentos basándose en UMAP y HDBSCAN.
2. **Modelo Guiado (Con Semillas):** Se introducen listas de semillas culinarias para orientar la formación de clústeres hacia conceptos de negocio significativos (e.g., vegano, postres, desayunos).

Para garantizar la representatividad estadística del modelo, se implementó un muestreo estratificado. Dividir el dataset original por cuartiles del tiempo de preparación (`minutes`) y muestrear proporcionalmente (10,000 recetas en total) asegura que tanto los platos de cocción lenta (asados, guisos) como los snacks ultrarrápidos (5 minutos) estén equilibradamente representados.

Métrica de Evaluación	Modelo No Supervisado	Modelo Guiado (S)
Coherencia de Tópicos (C_v)	0.42	0.57
Clústeres de Negocio Identificados	3 / 10	9 / 10
Proporción de Tópicos Ruidosos (Outliers)	28.4 %	14.2 %
Interpretabilidad Humana	Media-Baja (mezcla de ingredientes)	Alta (estilos de cocina)

Cuadro 1: Resultados Comparativos del Modelado de Tópicos en CulinaryRAG.

Los resultados de la Tabla 1 demuestran que el modelo guiado con semillas disminuye drásticamente el porcentaje de documentos asignados al tópico outlier (-1) y mejora la métrica de coherencia semántica. En el modelo puramente no supervisado, BERTopic tiende a agrupar recetas por ingredientes comunes y genéricos (ej. sal, pimienta), perdiendo la semántica del estilo de preparación. El uso de semillas forzó al espacio latente a estructurarse en torno a dimensiones de negocio culinarias, permitiendo el funcionamiento efectivo del *Topic-Aware Boost* en el recomendador.

5. Conclusiones y Trabajo Futuro

El desarrollo de CulinaryRAG demuestra la viabilidad de construir un recomendador de recetas conversacional altamente sofisticado y preciso utilizando tecnologías de minería de textos locales y personalizadas. Al rechazar el uso de herramientas rígidas prefabricadas, el proyecto evidencia un control granular sobre las etapas críticas de indexación, recuperación híbrida (RRF), procesamiento de tópicos y enrutamiento agéntica.

5.1. Principales Logros del Sistema

- **Precisión Semántica:** La combinación de RRF + Cross-Encoder resuelve satisfactoriamente las consultas complejas y las negaciones críticas (por ejemplo, recetas sin horno).
- **Eficiencia de Contexto:** El pipeline offline y el resumen TextRank garantizan un consumo de recursos en GPU/VRAM altamente optimizado, ideal para modelos de lenguaje locales.
- **Robustez Agéntica:** La decodificación restringida sobre los logits de Gemma-3 elimina las alucinaciones del router, logrando un flujo de control 100 % predecible.

5.2. Líneas de Trabajo Futuro

Como líneas de desarrollo futuro, se propone:

1. **Personalización de Usuario Histórica:** Integrar la base de datos de reseñas históricas del usuario para ajustar dinámicamente los pesos semánticos de RRF en base a perfiles alimenticios individuales precalculados de forma asíncrona.
2. **Optimización del Web Fallback:** Implementar un raspador (scraper) en tiempo real en el agente Web para extraer y estructurar automáticamente recetas procedentes de blogs externos, enriqueciendo la generación cuando la base de datos local sea insuficiente.